



上海科技大学
ShanghaiTech University

Lecture 04: Transformers

Zhuyu Lin
Spring, 2026

Outline



■ Language Modeling

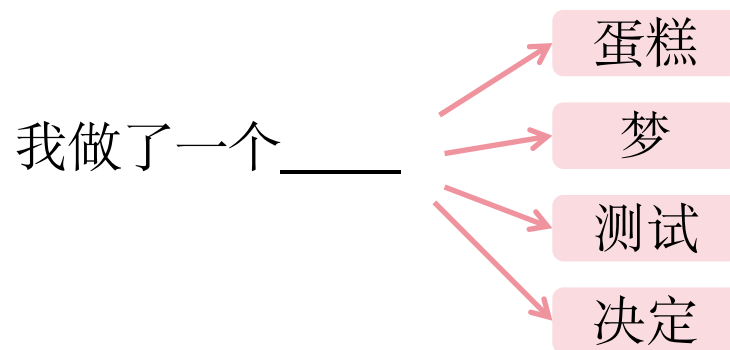
■ Transformer

- ☐ Self-Attention
- ☐ Positional Encoding
- ☐ Feed-Forward Network
- ☐ Residual Connections
- ☐ Layer Normalization
- ☐ Cross-Attention

Language Modeling



■ **Language Modeling** is the task of predicting what word comes next.



■ More formally: given a sequence of words $x_1, x_2, \dots, x_t$, compute the probability distribution of the next word x_{t+1} :

$$P(x_{t+1} | x_t, x_{t-1}, \dots, x_1)$$

where x_{t+1} can be any word in the vocabulary $V = \{w_1, w_2, \dots, w_{|V|}\}$

■ A system that does this is called a **Language Model**

Language Modeling



- You can also think of a Language Model as a system that **assigns a probability to a piece of text**
- For example, if we have some text $x_1, x_2, \dots, x_T$, then the probability of this text (according to the Language Model) is:

$$P(x_1, x_2, \dots, x_T) = P(x_1)P(x_2|x_1)\dots P(x_T|x_{T-1}, \dots, x_1)$$

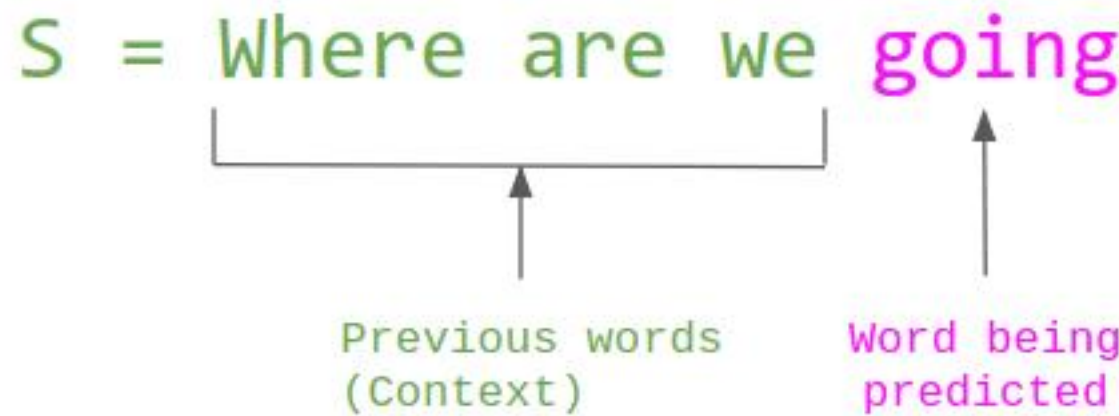
$$= \prod_{t=1}^T \underbrace{P(x_t|x_{t-1}, \dots, x_1)}$$

This is what our LM provides

Language Modeling



$$P(x_1, x_2, \dots, x_T) = P(x_1)P(x_2|x_1)\dots P(x_T|x_{T-1}, \dots, x_1)$$
$$= \prod_{t=1}^T P(x_t|x_{t-1}, \dots, x_1)$$



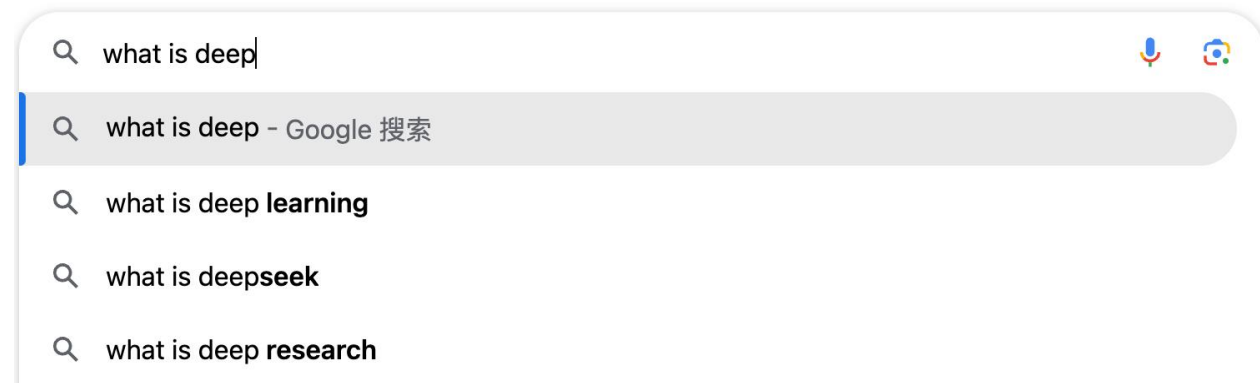
$$P(S) = P(\text{Where}) \times P(\text{are} \mid \text{Where}) \times P(\text{we} \mid \text{Where are}) \times P(\text{going} \mid \text{Where are we})$$

image source: <https://thegradient.pub/understanding-evaluation-metrics-for-language-models/>

Language Modeling



■ We use Language Models every day!





Language Modeling

■ We use Language Models every day!

你好，我是千问

向千问提问

✳ 任务助理

🔍 深度思考

🔍 深度研究

📄 代码

🖼 图像

☰ 更多

🌐

📎

📤



录音



PPT



音视频



文档



发现

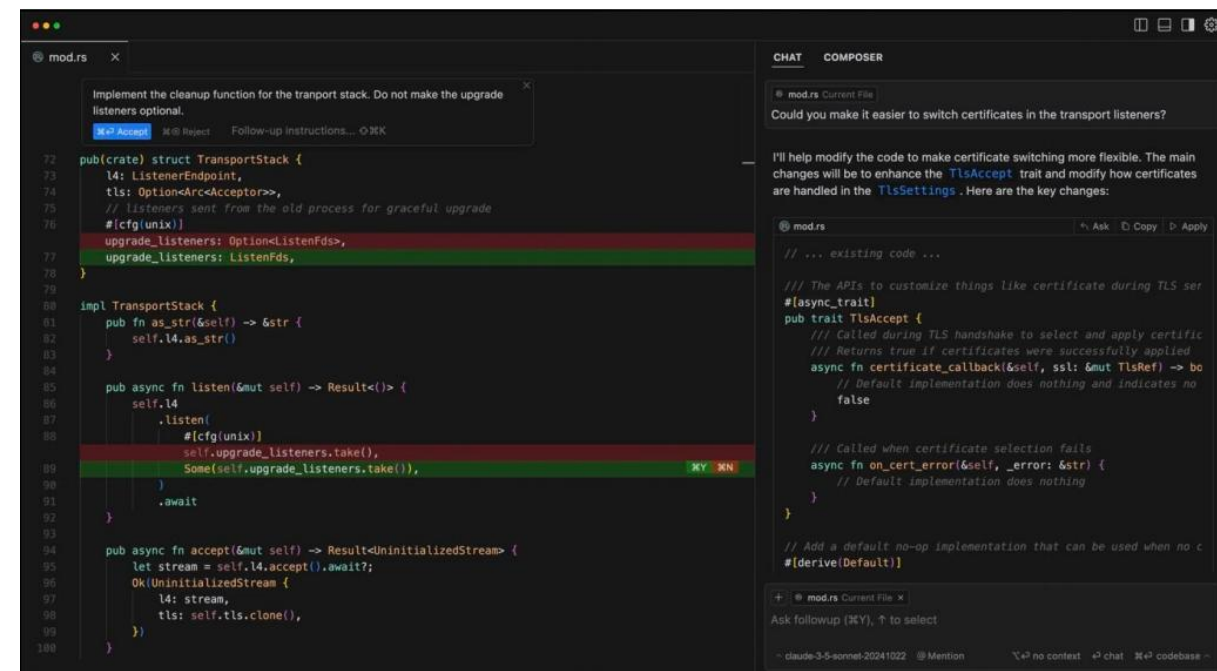
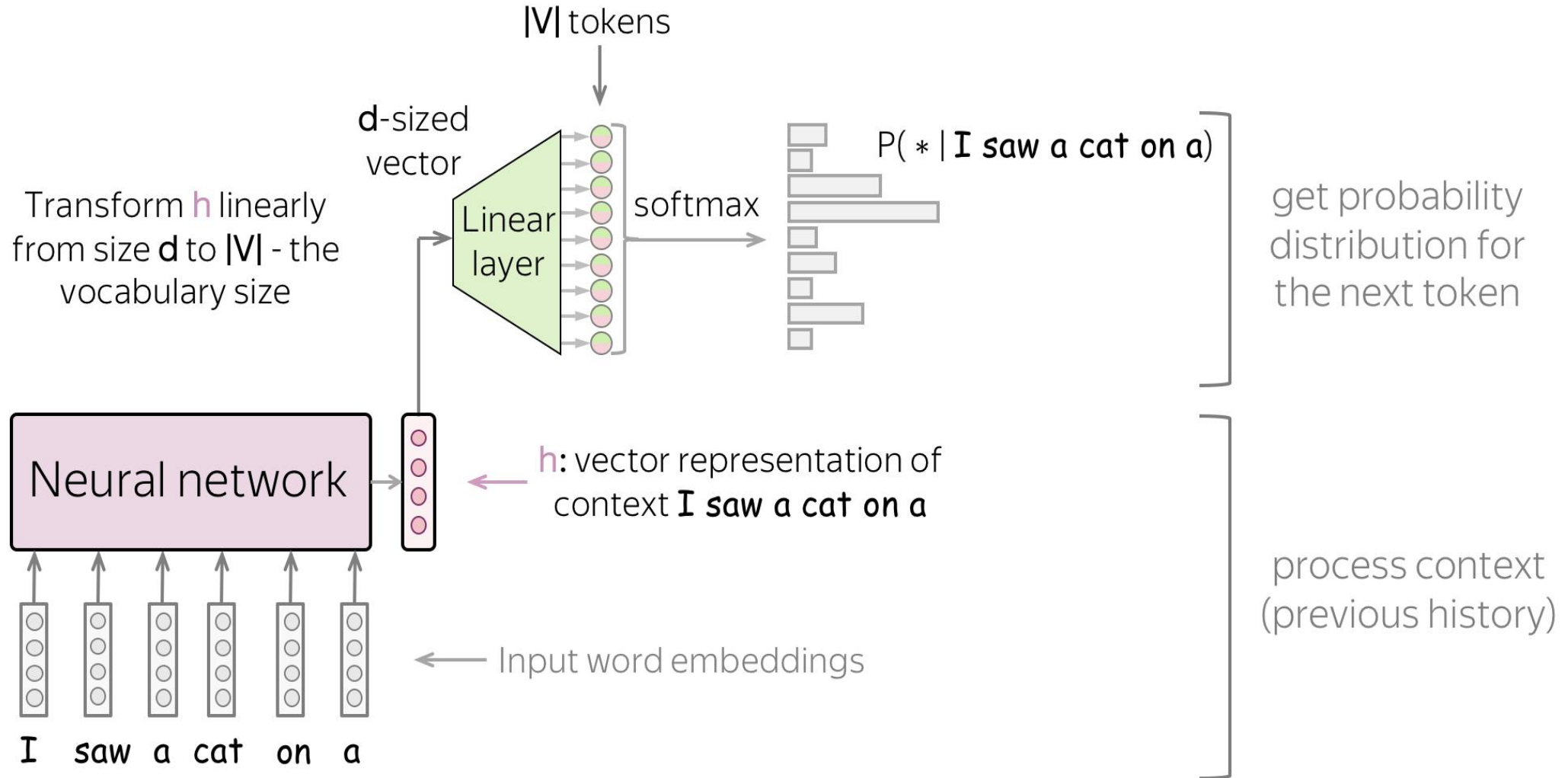


image source:

https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture04-rnnlm.pdf

Neural Language Model



Neural Language Model



A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

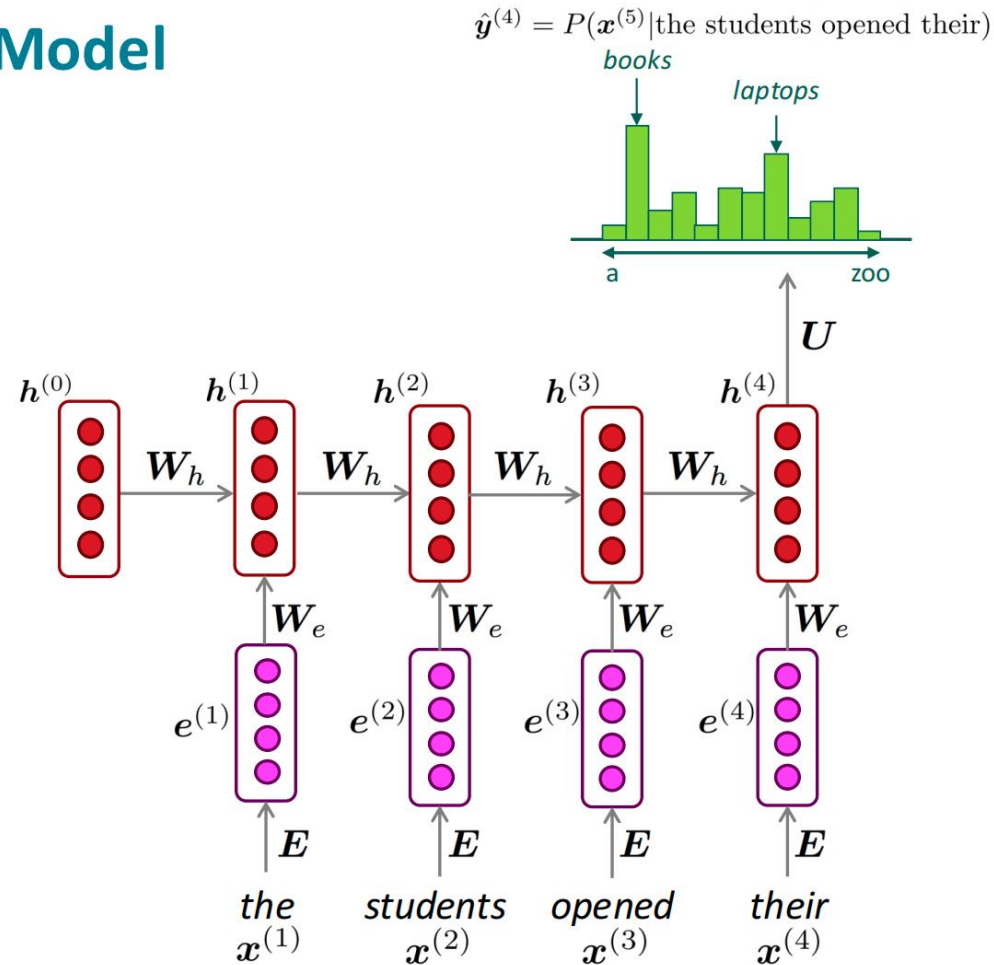
$h^{(0)}$ is the initial hidden state

word embeddings

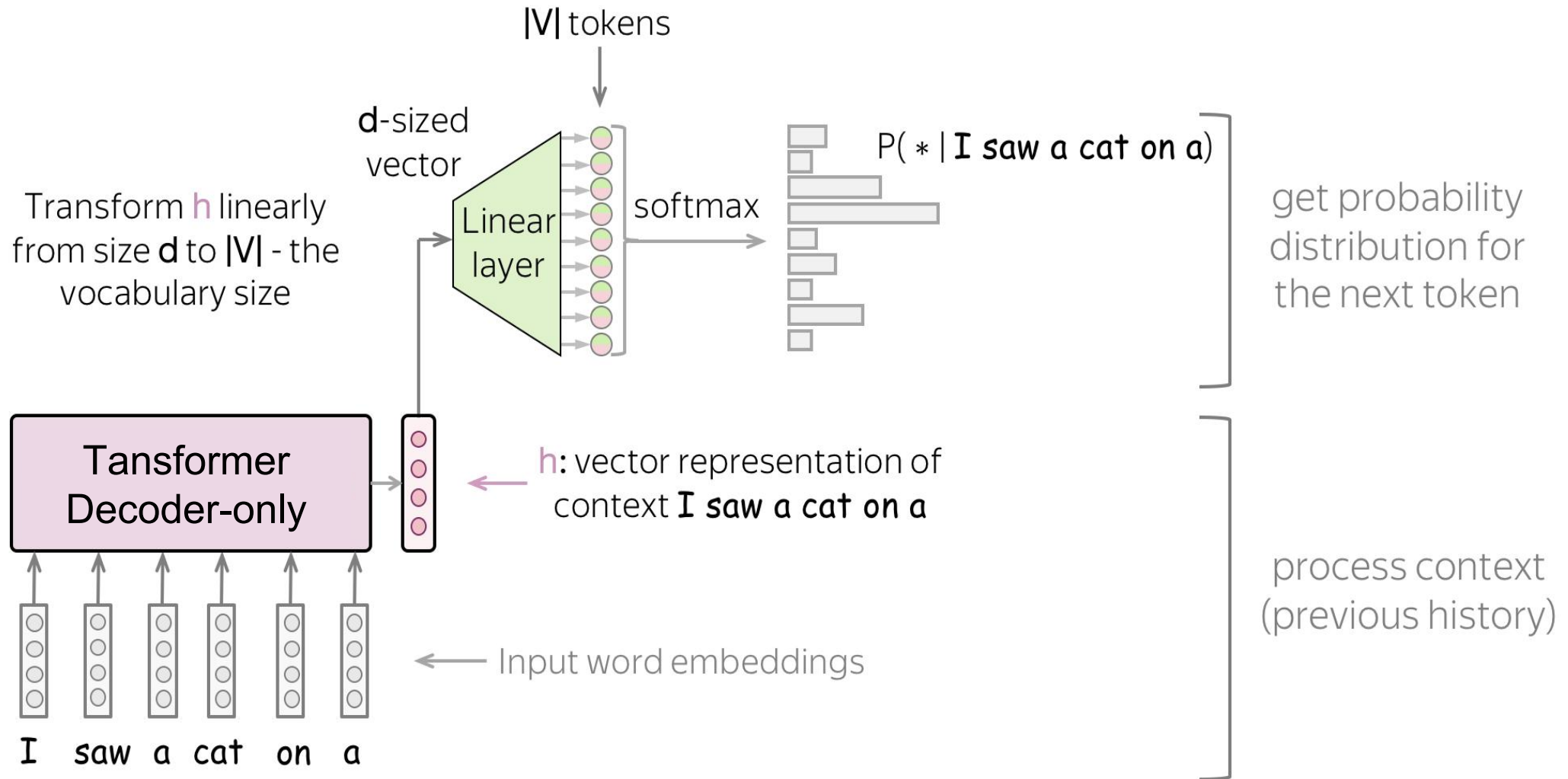
$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Neural Language Model



Transformer



上海科技大学
ShanghaiTech University

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

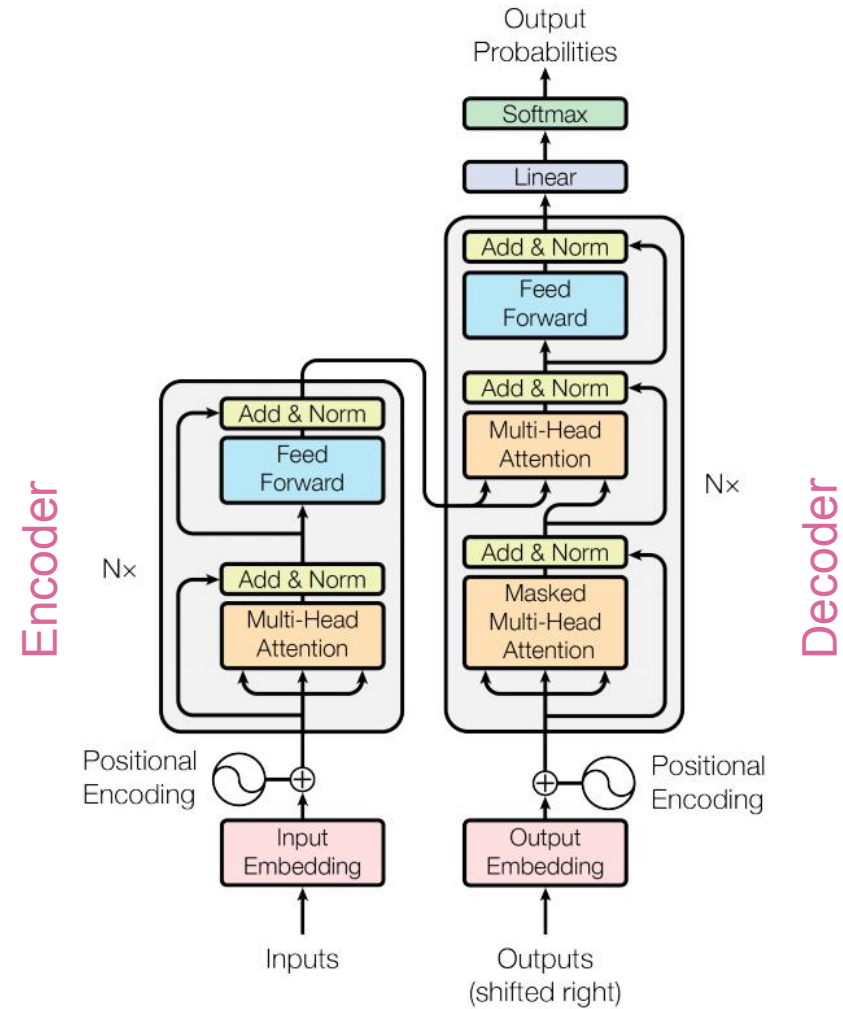
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

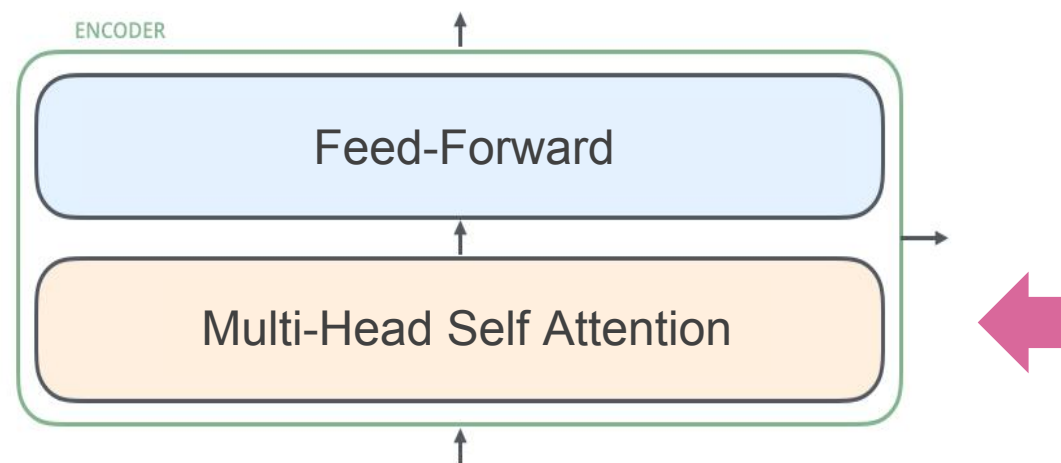
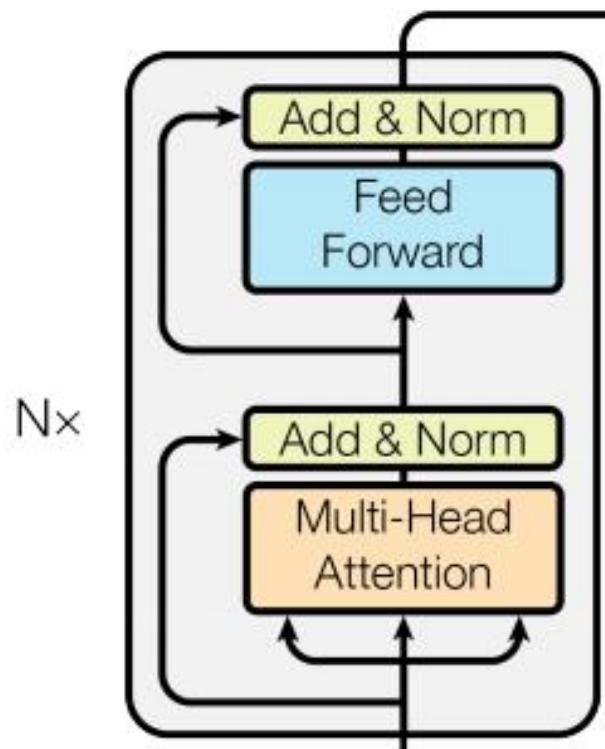


Vaswani et al., 2017. [Attention Is All You Need](#)

Transformer



■ Encoder



Self-Attention

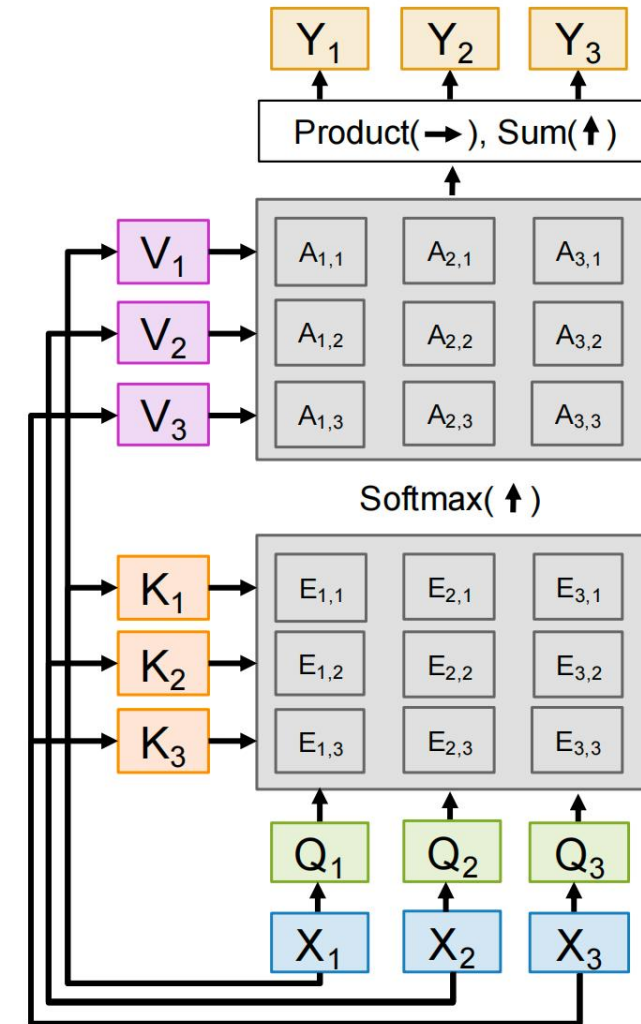


■ Cross-Attention

- Query (Q): comes from the Decoder
- Key (K) and Value (V): come from the Encoder

■ Self-Attention

- The Query (Q), Key (K), and Value (V) are all derived from the same input sequence.



Self-Attention

- Let $w_{1:n}$ be a sequence of words in vocabulary V
- For each w_i , let $x_i = E w_i$, where $E \in \mathbb{R}^{d \times |V|}$ is an embedding matrix.
- 1. Transform each word embedding with weight matrices W^Q , W^K , W^V , each in $\mathbb{R}^{d \times d}$

$$\begin{array}{ccc} \textcolor{violet}{q}_i = x_i W^Q & \textcolor{brown}{k}_i = x_i W^K & \textcolor{blue}{v}_i = x_i W^V \\ \text{queries} & \text{keys} & \text{values} \end{array}$$

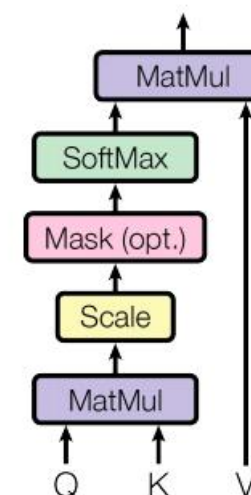
- 2. Compute pairwise similarities between keys and queries; normalize with softmax

Scaled Dot-Product $e_{i,j} = (\textcolor{violet}{q}_i^T \textcolor{brown}{k}_j) / \sqrt{d}$ $\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{j'} \exp(e_{i,j'})}$

- 3. Compute output for each word as weighted sum of values

$$o_i = \sum_j \alpha_{i,j} \textcolor{blue}{v}_j$$

Scaled Dot-Product Attention



Self-Attention



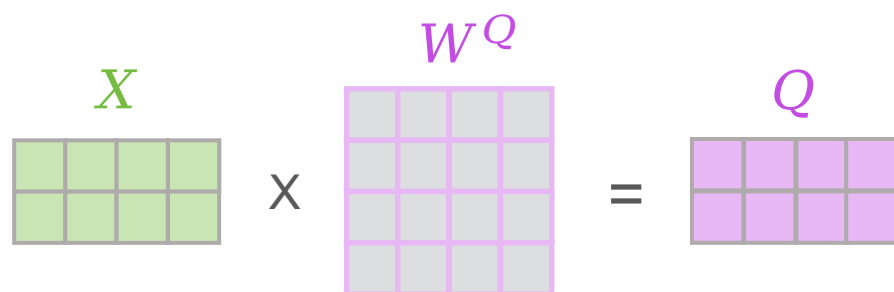
In matrices

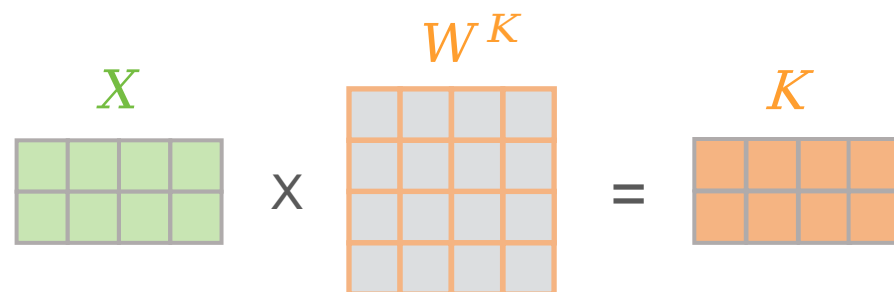
■ Let $X = [x_1, x_2, \dots, x_n]$ be the concatenation of input vectors.

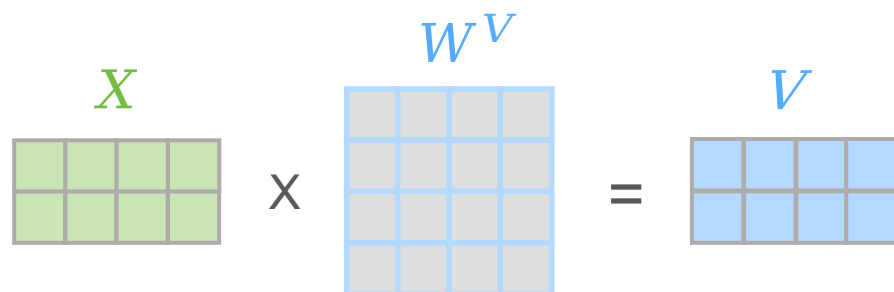
■ First, note that $Q = XW^Q \in \mathbb{R}^{n \times d}$, $K = XW^K \in \mathbb{R}^{n \times d}$, $V = XW^V \in \mathbb{R}^{n \times d}$

$$\text{output} = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

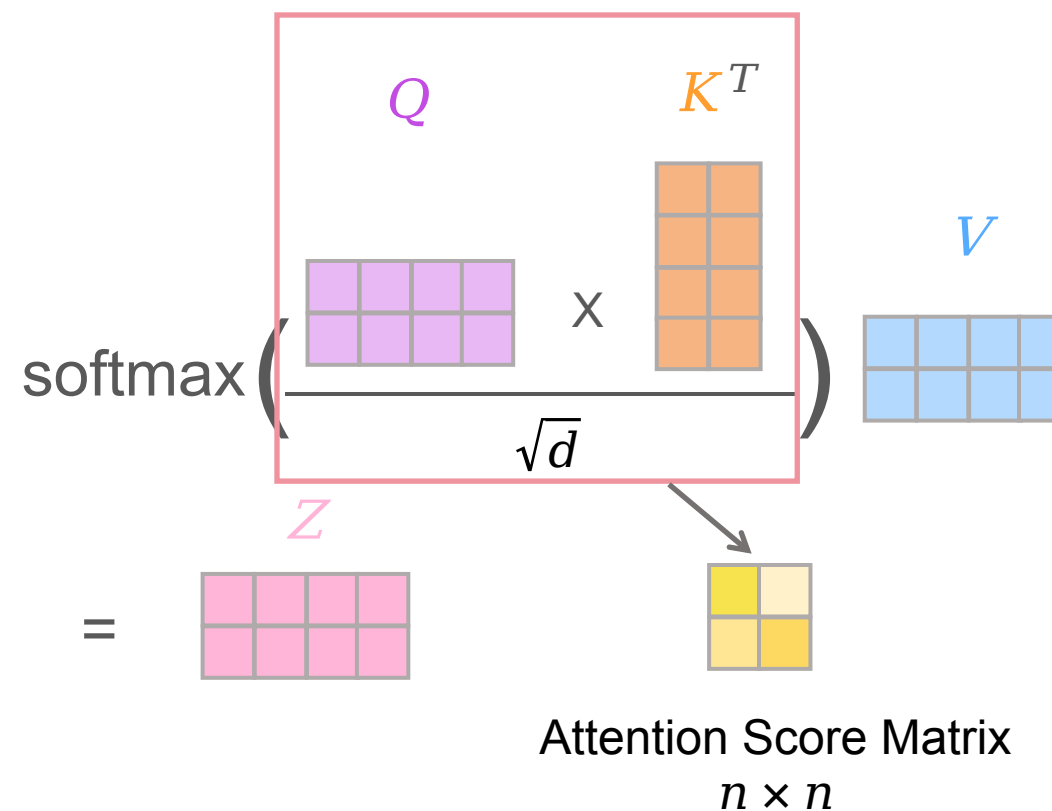
Self-Attention

$$X \times W^Q = Q$$


$$X \times W^K = K$$


$$X \times W^V = V$$


$$n = 2, d = 4, W^Q, W^K, W^V \in \mathbb{R}^{d \times d}$$

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d}}\right) = Z$$


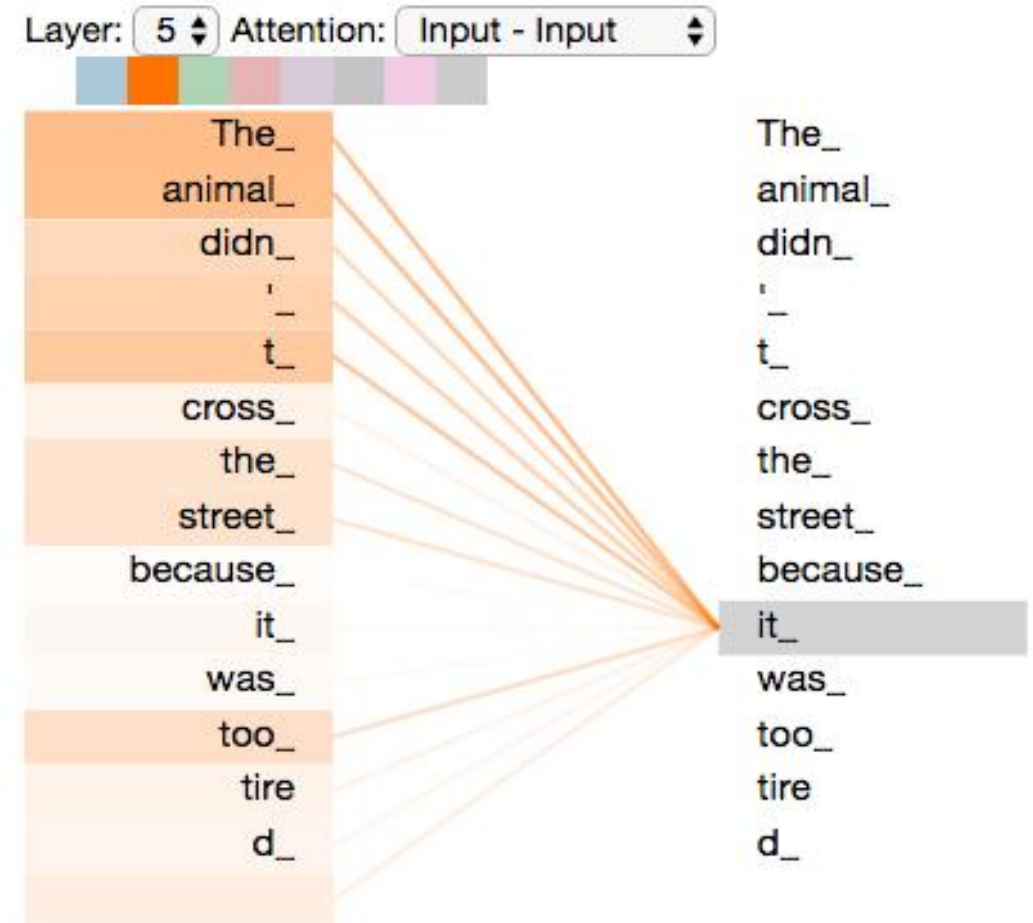
Attention Score Matrix
 $n \times n$

Self-Attention



■ Example: *The animal didn't cross the street because **it** was too tired.*

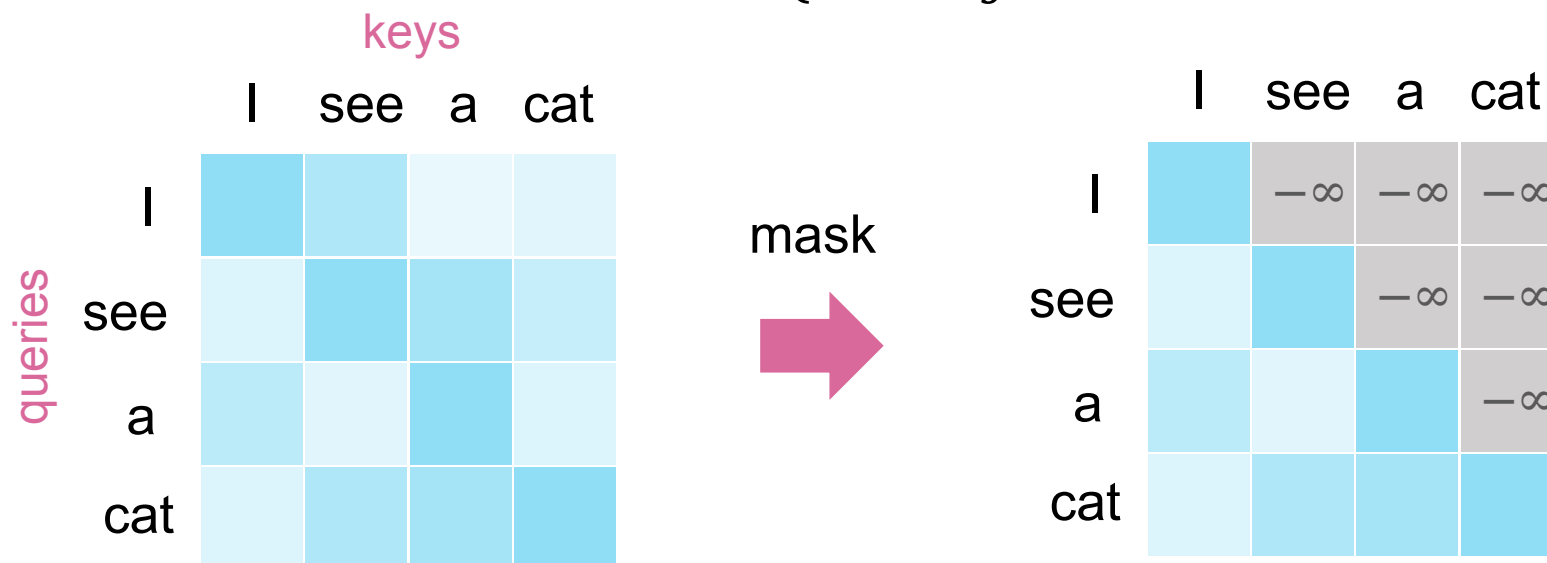
- What does “it” in this sentence refer to? Is it referring to the street or to the animal?
- Self-attention allows it to associate “it” with “animal”.



Masked Self-Attention

- Need to ensure we don't “look at the future” when predicting a sequence
 - Like in machine translation or language modeling
- To enable parallelization, we mask out attention to future words by setting attention scores to $-\infty$.

$$e_{ij} = \begin{cases} q_i^T k_j, & j \leq i \\ -\infty, & j > i \end{cases}$$

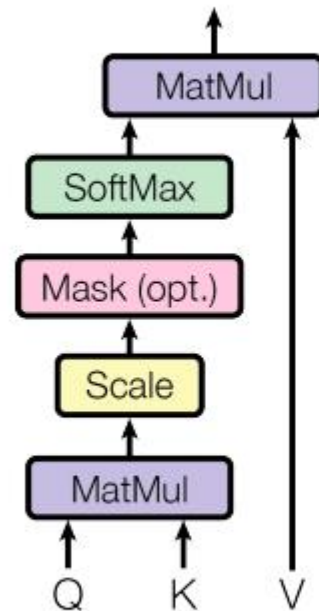


also called
causal self-attention

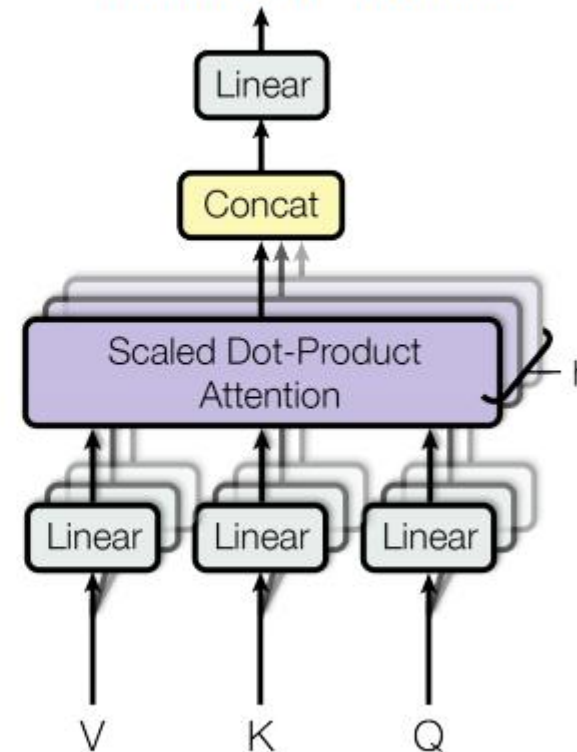
Multi-Head Self-Attention



Scaled Dot-Product Attention



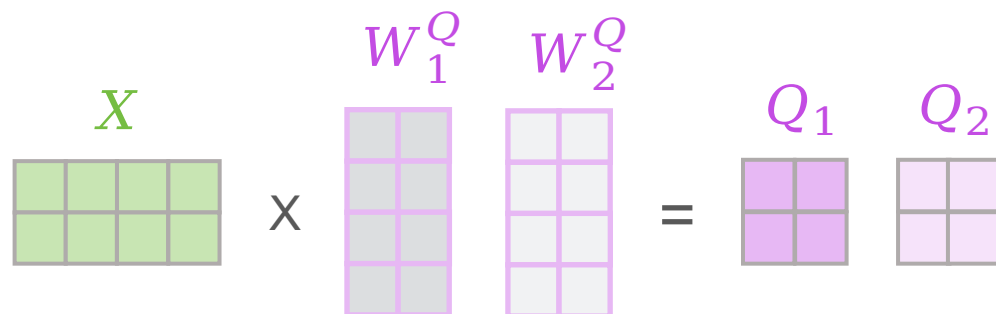
Multi-Head Attention

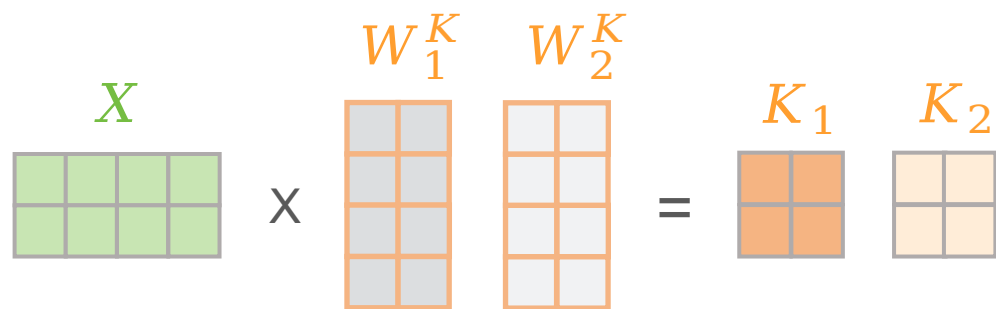


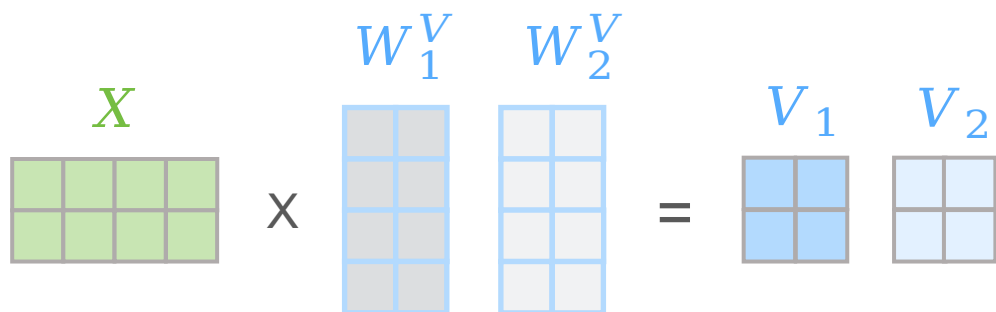
Vaswani et al., 2017. [Attention Is All You Need](#)

Multi-Head Self-Attention

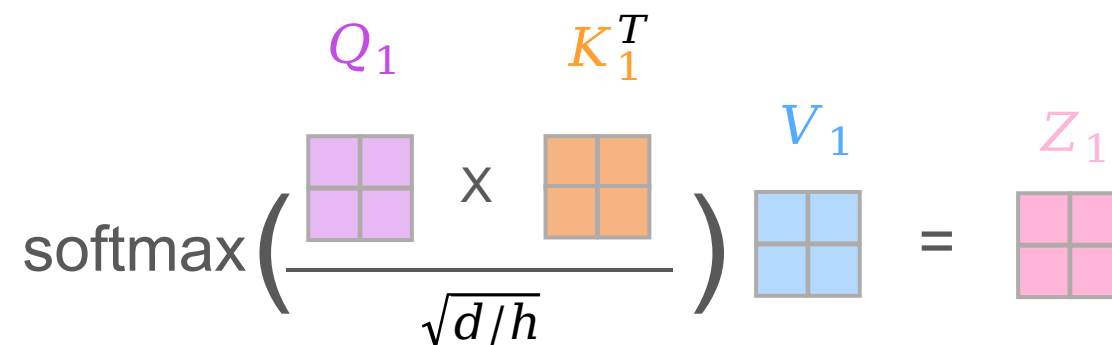
head₁ head₂

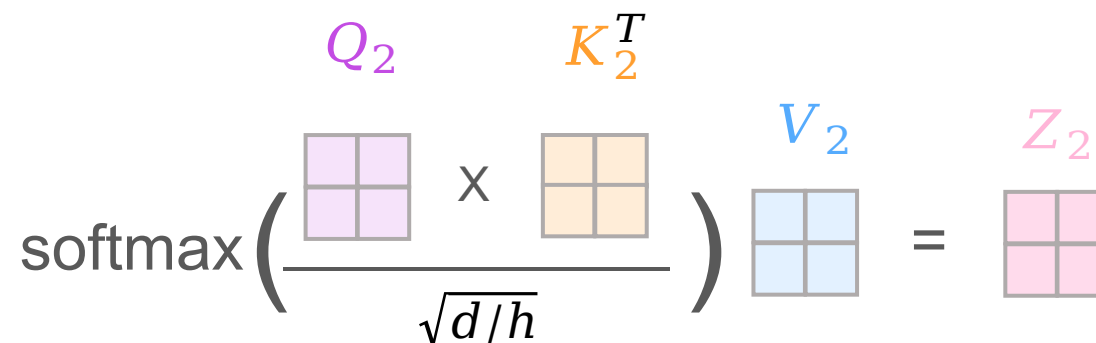
$$X \times W_1^Q \quad W_2^Q = Q_1 \quad Q_2$$


$$X \times W_1^K \quad W_2^K = K_1 \quad K_2$$


$$X \times W_1^V \quad W_2^V = V_1 \quad V_2$$


$$d = 4, h = 2, W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times \frac{d}{h}}$$

$$\text{softmax}\left(\frac{Q_1 \times K_1^T}{\sqrt{d/h}}\right) V_1 = Z_1$$


$$\text{softmax}\left(\frac{Q_2 \times K_2^T}{\sqrt{d/h}}\right) V_2 = Z_2$$


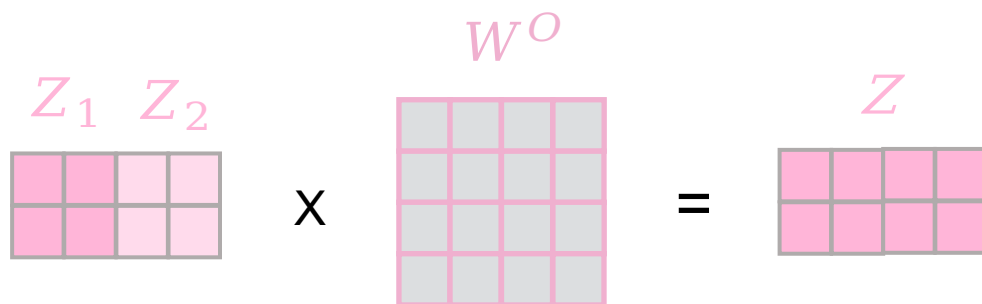
Multi-Head Self-Attention

$$Q_i = XW_i^Q, K_i = XW_i^K, V_i = XW_i^V$$

$$X \in \mathbb{R}^{n \times d}, i = 1, \dots, h, W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times \frac{d}{h}} \leftarrow \text{low rank}$$

$$Z_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d/h}}\right) V_i, Z_i \in \mathbb{R}^{n \times \frac{d}{h}}$$

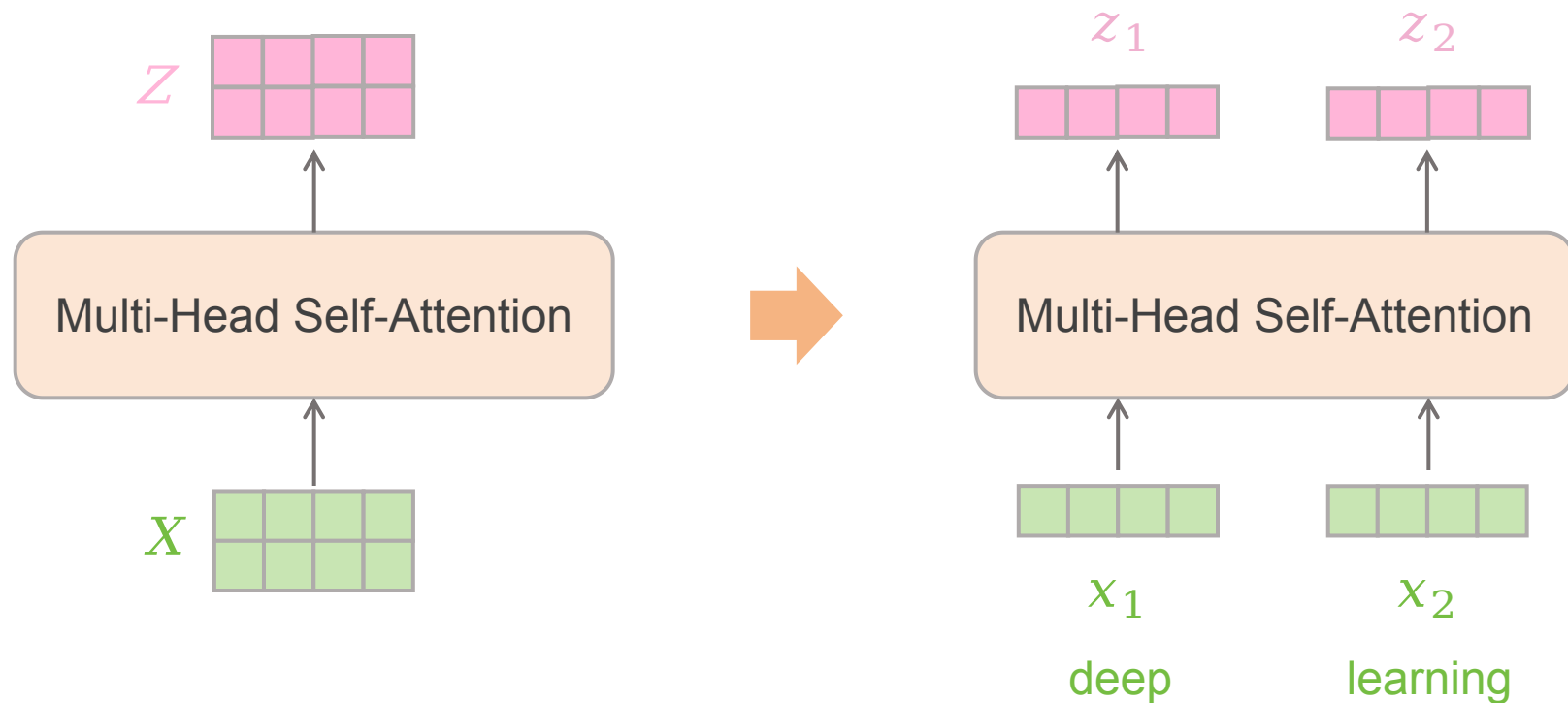
$$\text{MultiHead Attention Output: } Z = \text{Concat}(Z_1, \dots, Z_h) W^O, W^O \in \mathbb{R}^{(h \times \frac{d}{h}) \times d}$$



Multi-Head Self-Attention



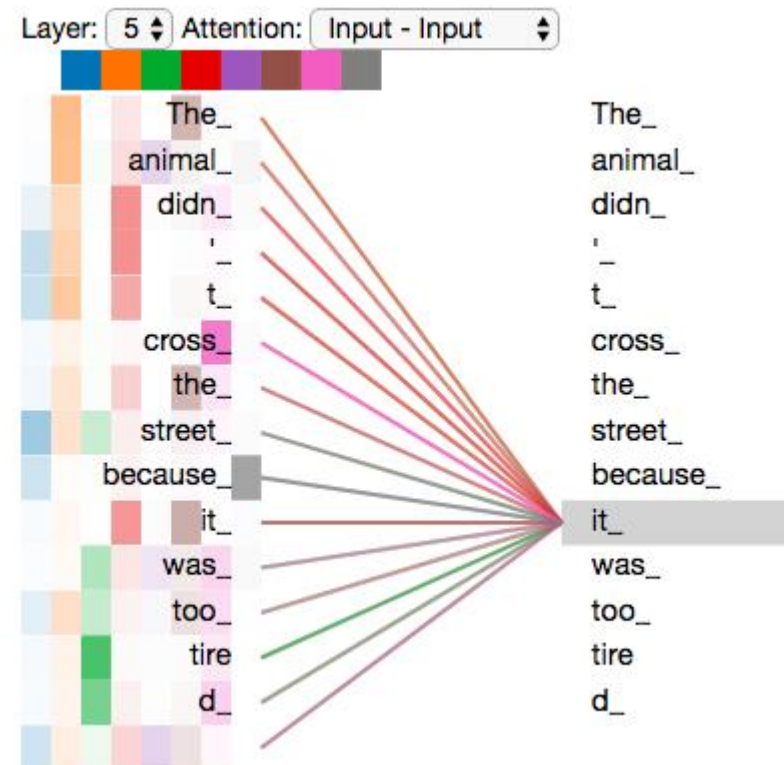
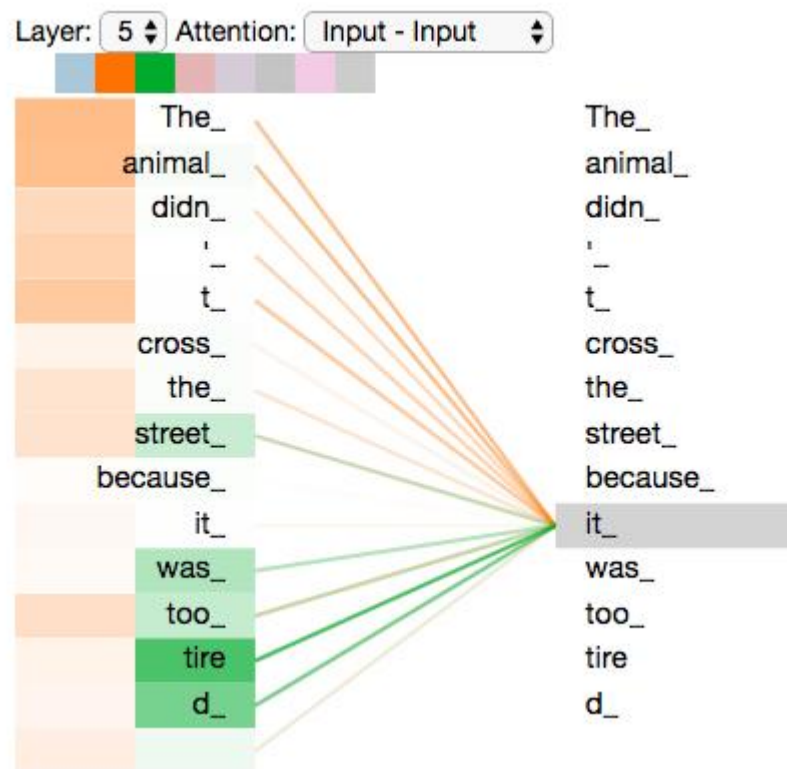
- Self-attention has identical input and output dimensions.



Why Use Multi-Head?



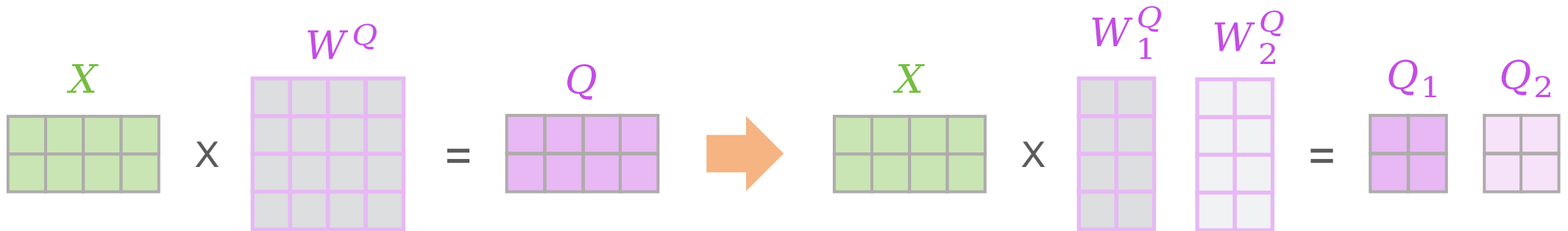
- Each head gets to “look” at different things (e.g., syntactic, semantic)



Why Use Multi-Head?



- Multi-head attention maintains **the same total parameters** as single-head attention.



- Multi-head self-attention is computationally efficient.



Positional Encoding

- Since self-attention and **doesn't build in order information**, we need to encode the order of the sentence in our keys, queries, and values.
- Consider representing each **sequence index** as a **vector**

$\mathbf{p}_i \in \mathbb{R}^d$, for $i \in \{1, 2, \dots, n\}$ are position vectors

- Add the p_i to our inputs:

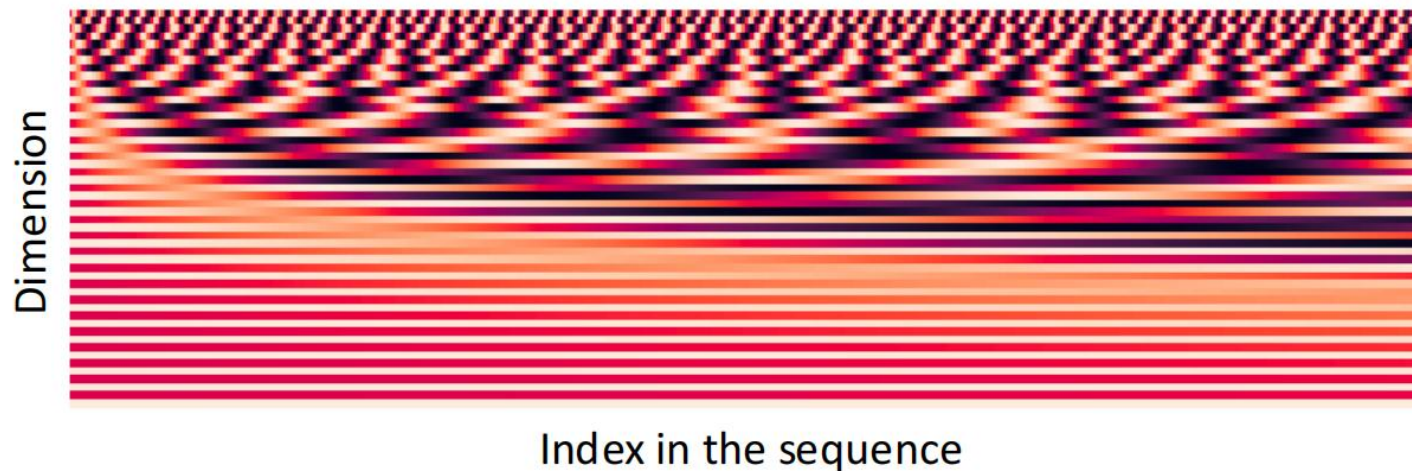
$$\widetilde{\mathbf{x}}_i = \mathbf{x}_i + \mathbf{p}_i$$

Positional Encoding



■ Sinusoidal positional encoding

$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



■ Pros

- Implicitly captures relative position relationships
- Maybe can extrapolate to longer sequences as periods restart

■ Cons

- Not learnable; also the extrapolation doesn't really work.

Positional Encoding



■ **Learned absolute position encoding:** Let all p_i be learnable parameters!

□ Learn a matrix $\mathbf{p} \in \mathbb{R}^{d \times n}$, and let each p_i be a column of that matrix.

■ **Pros:**

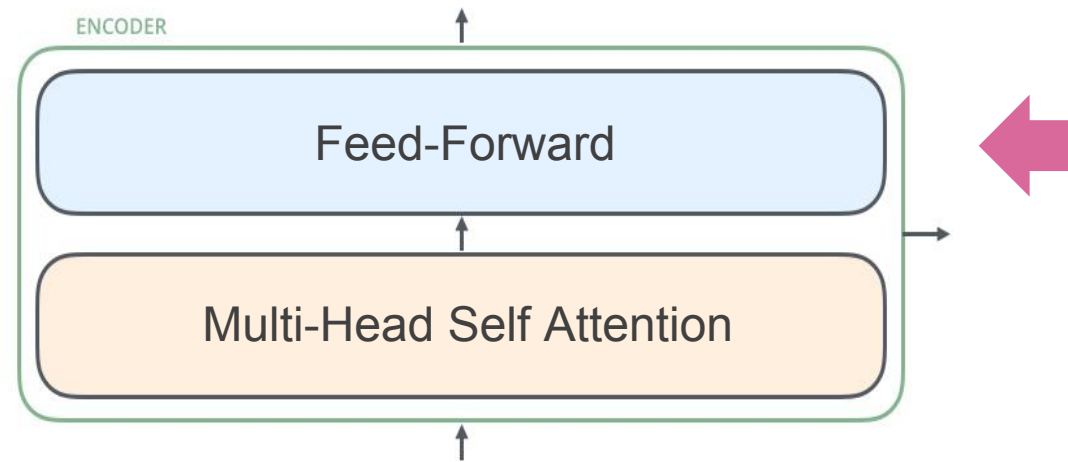
□ Flexibility: each position gets to be learned to fit the data

■ **Cons:**

□ Definitely can't extrapolate to indices outside $1, \dots, n$

■ Most modern LLMs (e.g., LLaMA) now use **RoPE** (Su et al., 2021) or other advanced positional representations.

Feed-Forward

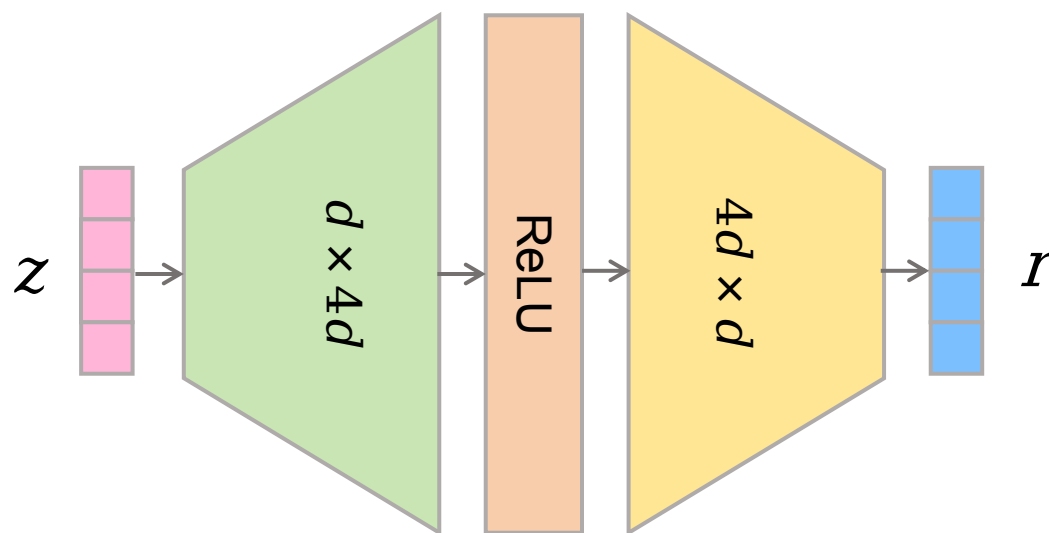


Feed-Forward

■ Position-wise Feed-Forward Networks

$$\text{FFN}(x) = \text{MLP}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

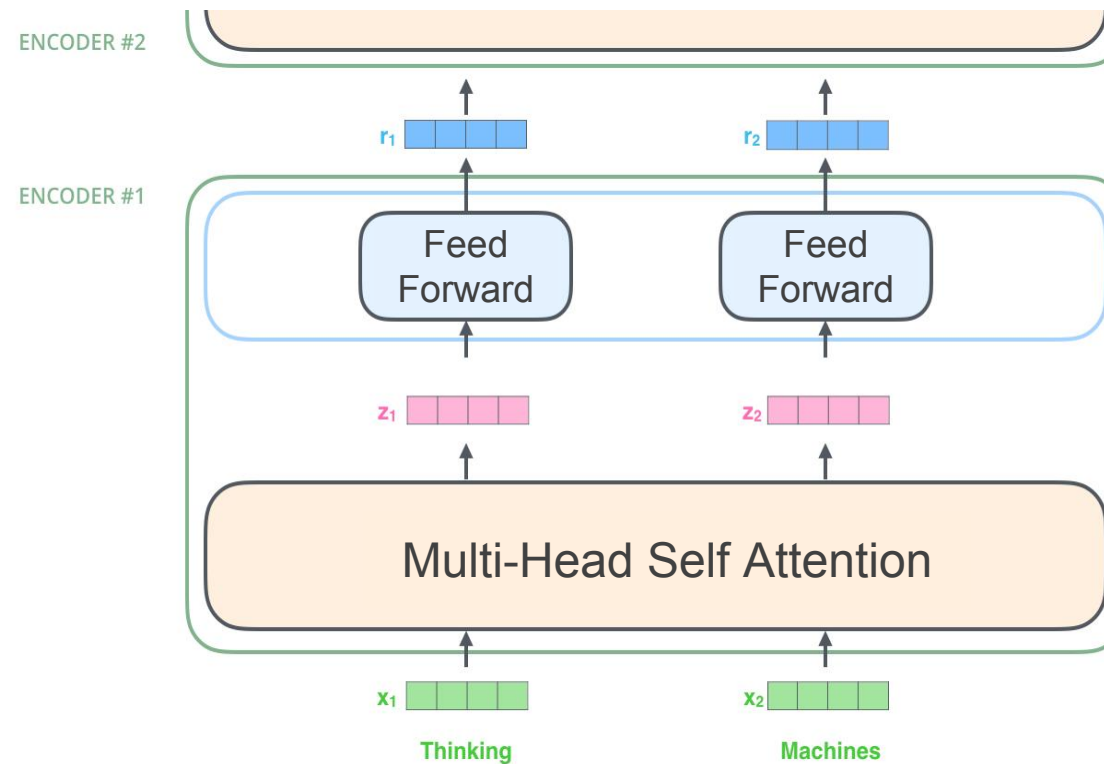
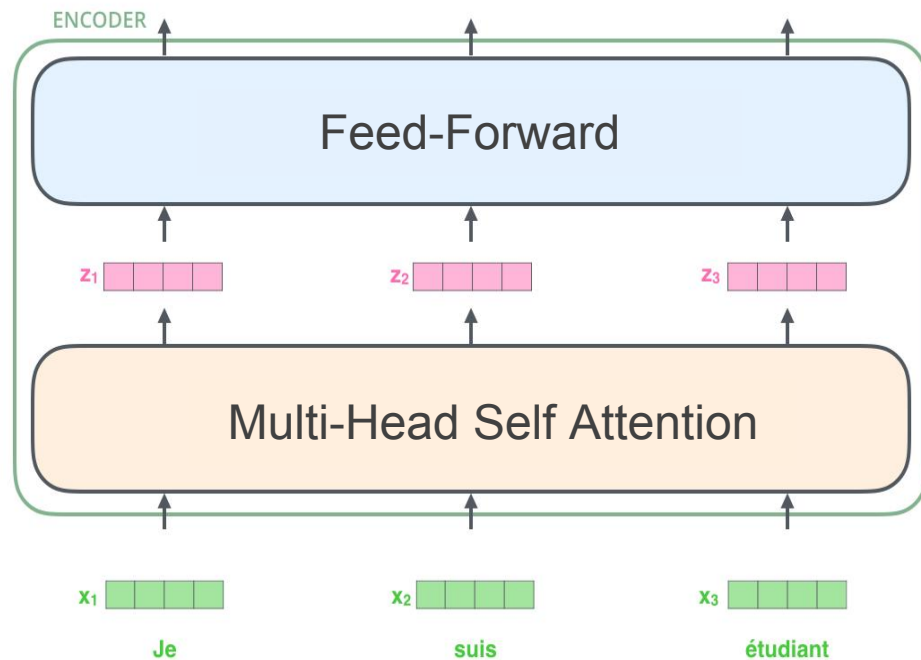
$$W_1 \in \mathbb{R}^{d \times 4d}, W_2 \in \mathbb{R}^{4d \times d}$$



Feed Forward



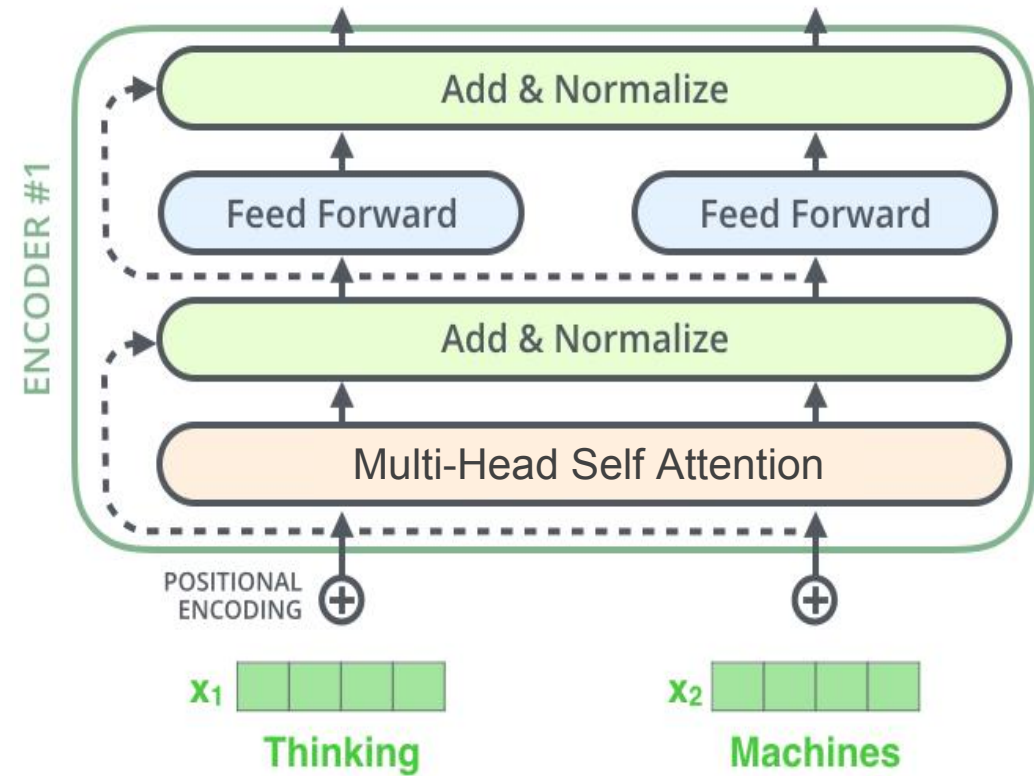
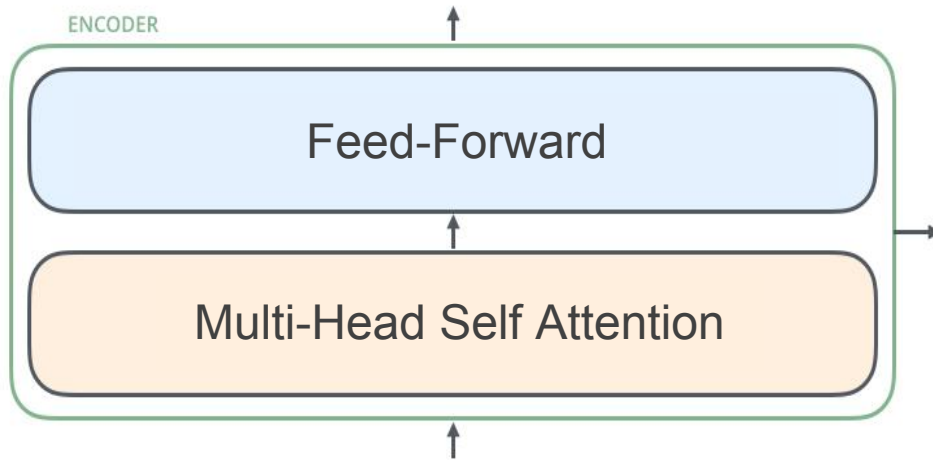
- Applied to each position separately and identically.



Add & Norm



- Residual connections
- Layer normalization



Residual Connections



■ **Residual connections** are a trick to help models train better (He et al., 2016)

□ Instead of $X^{(i)} = \text{Layer}(X^{(i-1)})$ (where i represents the layer)



□ We let $X^{(i)} = X^{(i-1)} + \text{Layer}(X^{(i-1)})$ (so we only have to learn “the residual” from the previous layer)



□ Gradient is great through the residual connection; it's 1!

□ Enabling deep architectures

- He et al., 2016. [Deep Residual Learning for Image Recognition](https://arxiv.org/abs/1512.03385)
- https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture05-transformers.pdf



Layer Normalization

■ **Layer normalization** is a trick to help models train faster (Ba et al., 2016)

■ High-level Idea: Learn parameters that let us **scale / shift** the input data

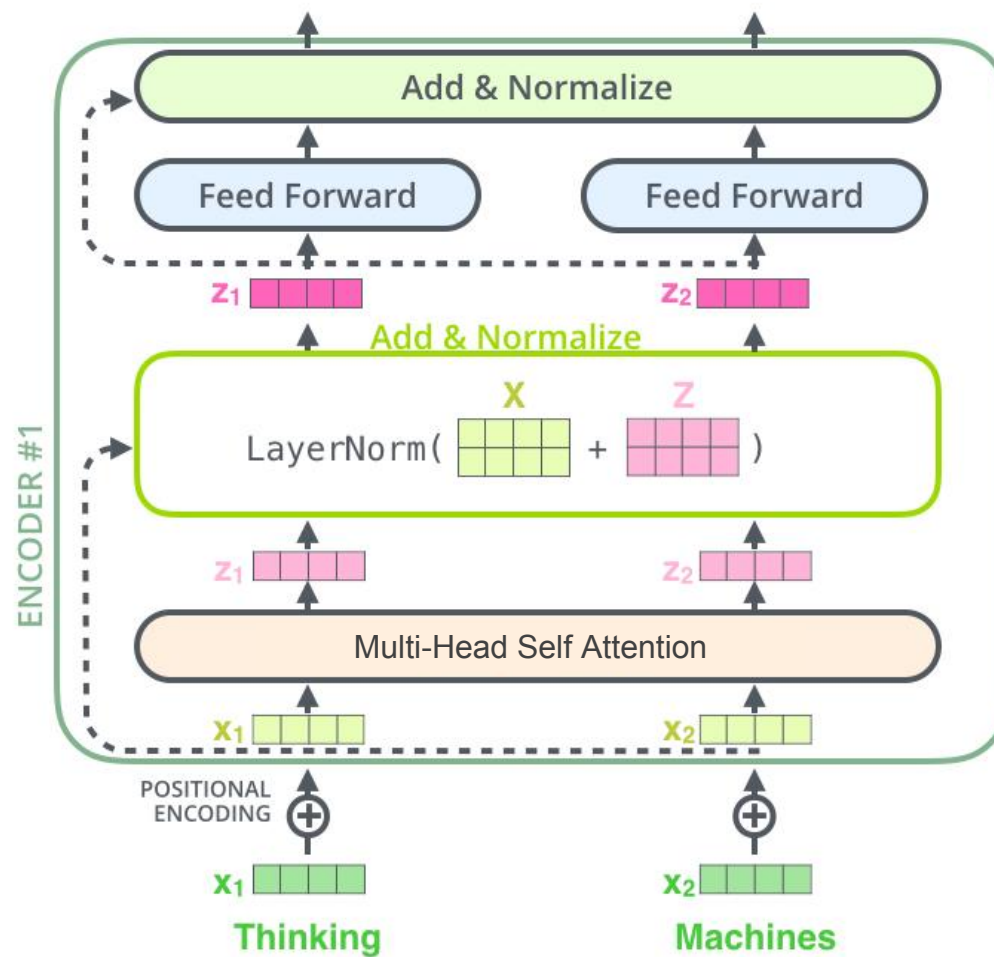
Let $x \in \mathbb{R}^d$ be an individual (word) vector in the model

1. Normalize input data x
2. Scale / shift using learned parameters: $\gamma \in \mathbb{R}^d, \beta \in \mathbb{R}^d$

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i \quad \sigma = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 \quad \mu, \sigma \in \mathbb{R}$$

$$\text{output} = \frac{x - \mu}{\sqrt{\sigma + \varepsilon}} \times \gamma + \beta$$

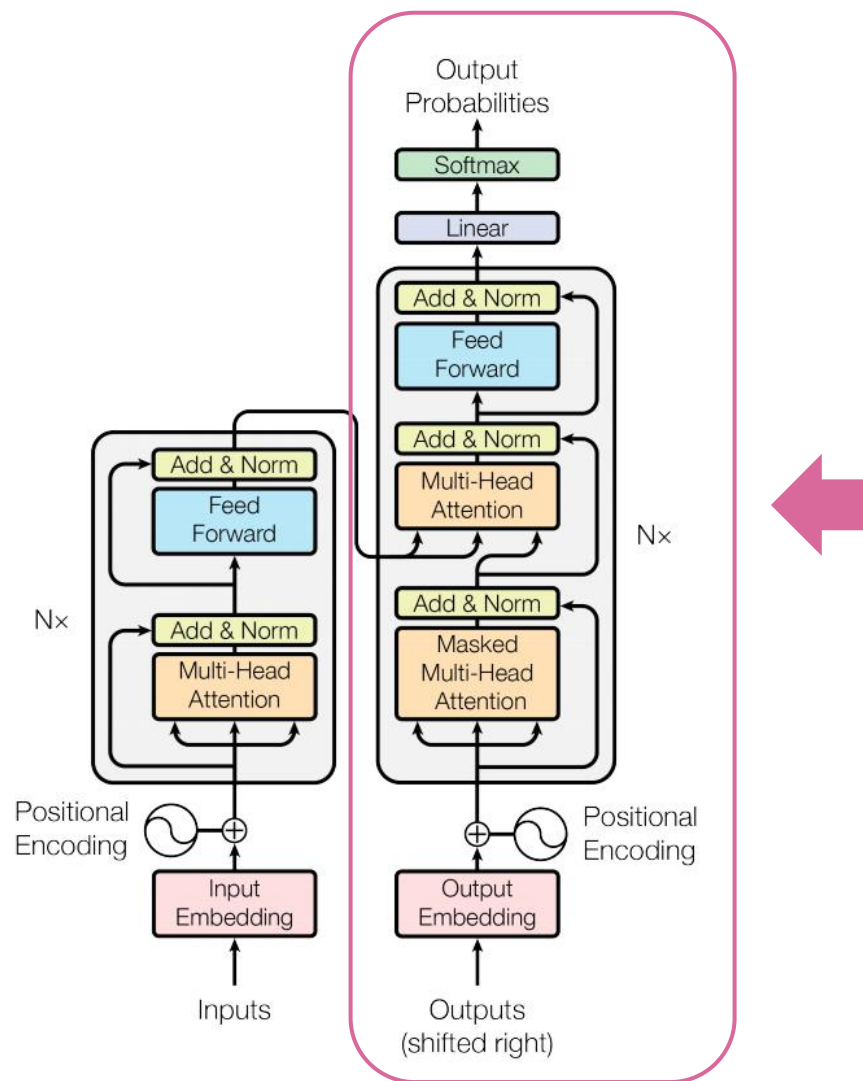
Add & Norm



Transformer



■ Decoder

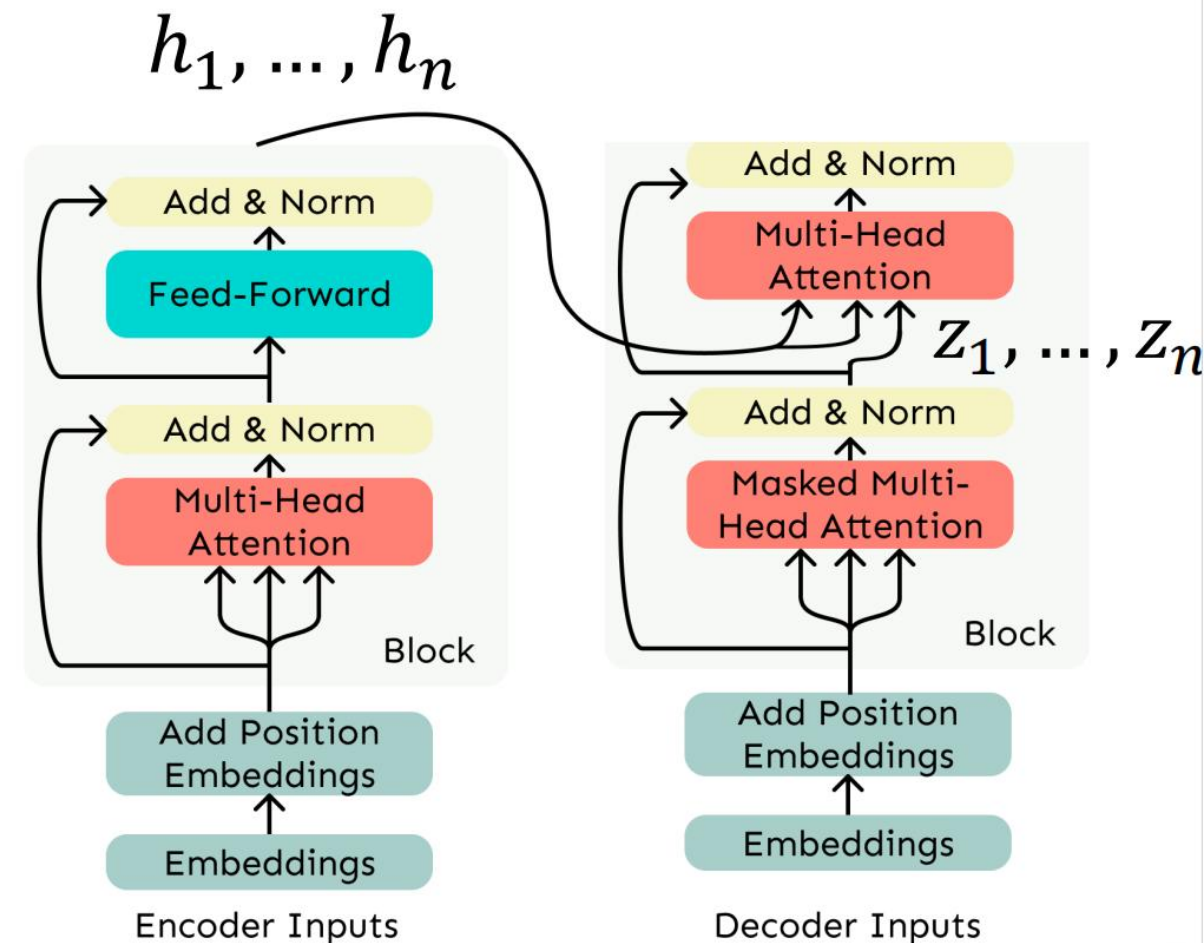


Cross-Attention

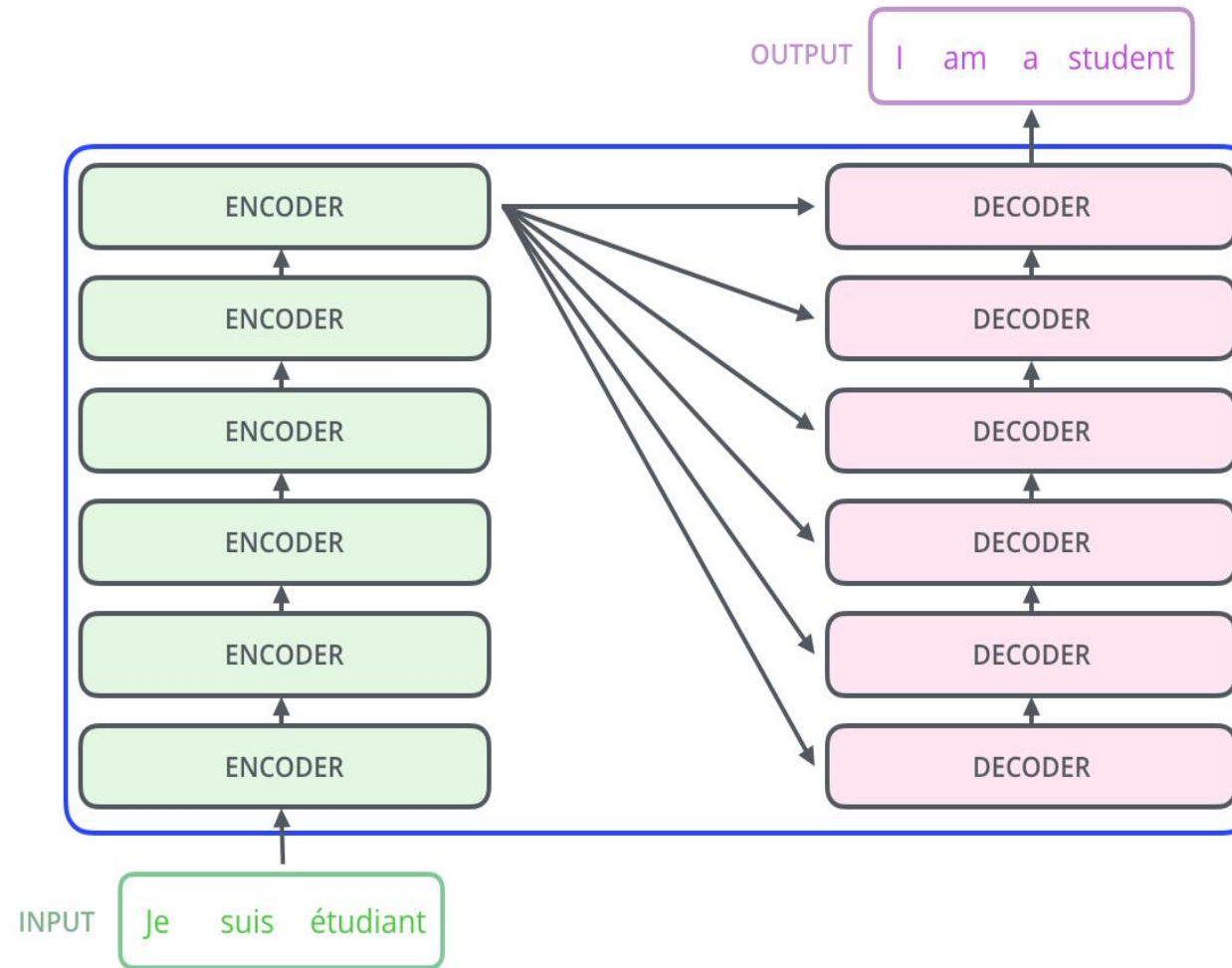
- Let $h_1, h_2, \dots, h_n$ be output vectors from the Transformer encoder, $h_i \in \mathbb{R}^d$
- Let $z_1, z_2, \dots, z_n$ be input vectors from the Transformer decoder, $z_i \in \mathbb{R}^d$

$$q_i = z_i W^Q$$

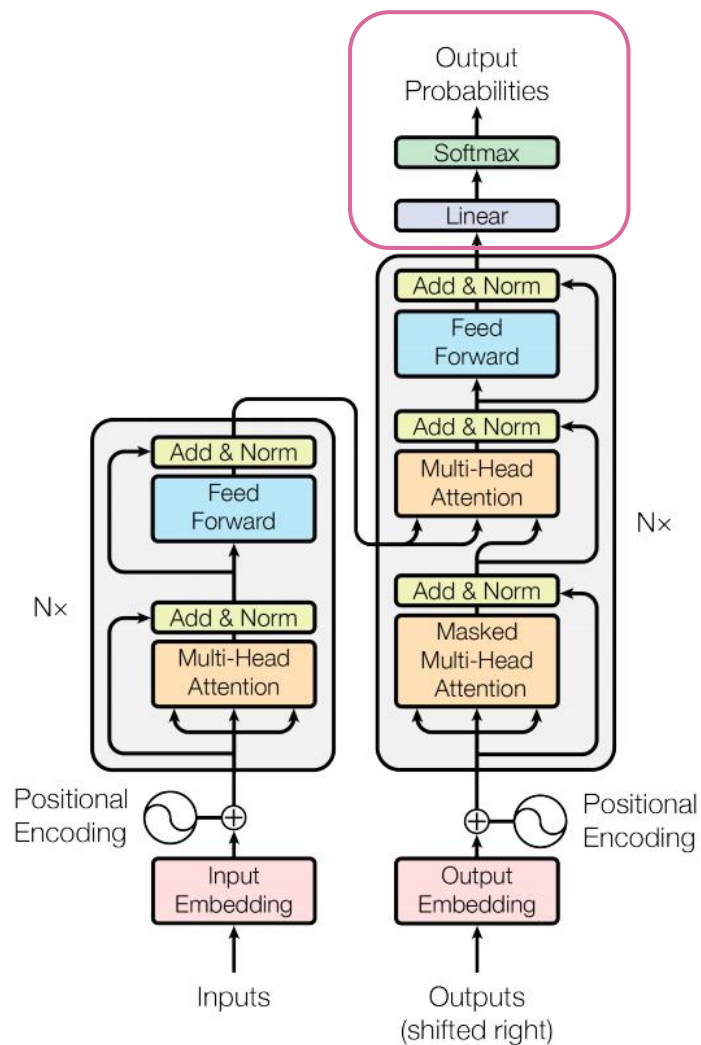
$$k_i = h_i W^K, v_i = h_i W^V$$



Cross-Attention



The Final Linear Layer



Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(**argmax**)

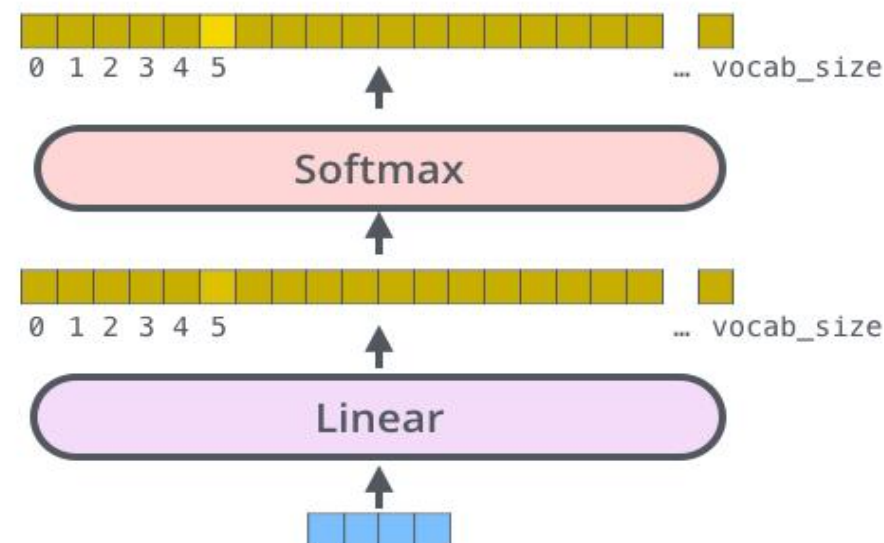
log_probs

logits

Decoder stack output

am

5



<https://jalammar.github.io/illustrated-transformer/>

Transformer

